为什么数组不能整体赋值：左值判定的底层逻辑

嵌入式 Linux 手记　2026-10-13　公众号文章 · 原链接 mp.weixin.qq.com

先看四行声明和两行赋值。哪一行编译不过?

```
struct Buf { int data[3]; };

int a[3] = {1, 2, 3};
int b[3];
struct Buf s1, s2;

b  = a;     // ①
s2 = s1;    // ②
```

① 报错，gcc 给出的原文是 `error: assignment to expression with array type`；② 通过，`s2` 得到 `s1` 的完整副本。同样是"一整块内存"，同样 12 字节，待遇天差地别。这篇把判定规则挖到底层，最后顺手给你一个让数组"复活"整体赋值能力的正统手法。

一、三个流行解释，都不成立

解释一："数组太大，复制开销高，所以语言禁止。"不成立：`int[3]` 才 12 字节，比多数结构体小得多，而几百字节的结构体赋值畅通无阻。

解释二："数组名是指针，指针赋值会改指向，危险。"方向反了：真正的指针变量恰恰可以随便赋值；反而是"最像指针"的数组名不行。

解释三："编译器懒得实现。"不成立：结构体整体赋值从 C89 起就是标准语义，三十多年了。

三个都倒掉之后，剩下的才是真相：这个问题与大小无关、与性能无关，它是类型系统的一条明文规则。要读懂这条规则，先得认识一个名字。

二、可修改左值：赋值号左边资格审查

标准对赋值的约束只有一句话：赋值运算符的左操作数必须是可修改左值（C11 §6.5.16）。"可修改左值"的定义写在 C11 §6.3.2.1，原文列了排除项，第一条就是数组类型。

为什么把数组排除在外？回到赋值要做的事：往一块已经存在的存储里写数据。所以左边必须"指得着一处可写的内存"。上一篇刚拆过形参，现在轮到表达式：**一个表达式能不能被赋值，取决于它代表"一块内存"，还是只代表"一个值"**。

用四要素的语言说：变量名、`*p`、`a[i]`、`s.member`、结构体变量，这些表达式背后都有属于自己的一块存储，写它们就是写那块存储；而数组名在表达式里代表的是首元素的地址值（篇 1 的退化规则），地址值是个"值"，和 `0x1000` 这个数字没有本质区别，你没法把 12 字节塞进一个数值里。

有个细节很有意思：gcc 的报错原文 `assignment to expression with array type`，结尾落在 array type 两个词上。报错里保留了"数组类型"，恰好证明左操作数 `b` 此刻没有退化成指针：赋值号左操作数必须以本来面目接受可修改左值资格审查（C11 §6.5.16p2），数组在这一关就被拦下，根本轮不到退化那一步。



图：`b = a;` 的编译器判定过程：保持数组身份接受资格审查，随后打回



图：左值判定一览：左边一列有自己的一块存储，可写；右边一列只是"值"，无处可写

对照着看一遍判定表：

表达式	可作左值	原因
`int x;` 的 `x`	可以	变量有存储
`*p`	可以	解引用后指向真实对象
`a[i]`	可以	等价 `*(a+i)`，指向具体元素
`s.member`	可以	成员是对象的一部分
`struct` 变量	可以	表达式代表聚合对象本身，语义为逐成员拷贝
数组名 a	不行	可修改左值定义明确排除数组类型
`&a`	不行	地址常量，无自身存储
`a + b`、`42`	不行	表达式的结果值（右值）

结构体之所以能整体赋值，是因为结构体表达式不代表地址，它就代表那个聚合对象本身，赋值语义被定义为逐成员拷贝，从 C89 沿用至今。数组在语言演化的另一条路上（篇 1 讲过的"表达

式中按地址处理")，从未获得过这种"作为整体对象"的表达式身份。

三、工程现场：让数组"搭便车"的正统手法

规则记住了，活还是要干。数组复制有两条常规路径：逐元素循环，或者 `memcpy`（不知道元素类型布局时，`memcpy(b, a, sizeof b)` 是唯一安全解）。

想要更多？把数组包进结构体：

```
typedef struct { int data[3]; } Buf;

Buf make(void)
{
    Buf b = {{1, 2, 3}};
    return b;               // 结构体按值返回：数组搭上了便车
}

Buf x = make();             // x.data 就是那个"被返回的数组"
```

结构体能整体赋值、按值传参、按值返回，数组包进去之后，这三种能力全部继承。C 语言没有"返回数组"的语法，这是公认的正统替代方案。Linux 内核里就有现成例子：`uuid_be` 的定义是 `typedef struct { __u8 b[16]; } uuid_be;`，16 字节的数组裹上结构体外壳，整个 UUID 就能按值传递和赋值了。

写代码前的自查口诀也随之出炉：落笔 `=` 之前问一句，左边是一个"能往里放东西的对象"，还是一个"值"？前者放行，后者编译器会替你拦下。这句话反过来还能解释一堆报错：`&a = x`（地址常量）、`"hello" = p`（字面量）、`getchar() = c`（函数返回值），全是同一款审查。

写在最后

把这篇压成五条：

1. 赋值要求左边是可修改左值；C11 §6.3.2.1 的定义把数组类型明确排除在外。b = a 报错不是性能取舍，是类型系统的明文规则。
2. 数组名在表达式里代表地址值（退化规则，篇 1），地址值没有存储，写不进 12 字节。门牌号不是房子，门牌号上也贴不了春联。
3. 结构体能整体赋值，因为它的表达式就代表聚合对象本身，语义为逐成员拷贝，C89 起标准化。

4. 数组复制的正解：逐元素循环、memcpy；想要赋值、传参、返回全套能力，用结构体包装，内核的 uuid_be 是现成范例。
5. 自查口诀：= 左边必须是"能往里放东西的对象"。变量名、*p、a[i]、s.member、结构体变量都行；数组名、&a、常量、表达式结果都不行。

下一篇换把趁手的武器：`qsort` 为什么只靠一个函数指针就能排任何类型？把"把行为当数据"的回调机制从 `qsort` 拆到状态机表驱动，函数指针不是炫技，是 C 语言的解耦神器。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。